FOUR STUDY TRACKS · ONE COMPLETE SYSTEM

Team Defense Guide

Checkable facts, spoken parts, exercises, examiner questions and a 15-minute rehearsal for the whole team.

The project in one minute

Calcio Champagne is a reproducible football data system that asks whether incoming transfer spending accompanies an improvement in next season's recorded attacking output, and whether attacking output tracks same-season league finish. It combines pinned CSV acquisition, PostgreSQL, a Python processing pipeline, measured infrastructure and a browser application. The primary model improves modestly over a zero-change baseline. A separate comparison reports score-derived GF/GA/GD and league ranks, observed shots, shots on target and corners, plus descriptive Poisson attack/defense ranks. The source has no provider xG; an SOT-based conversion proxy is explicitly experimental. A goalkeeper cross-check and red-card association are presented with their coverage and limits. The primary Transfermarkt-derived dataset's licence remains unconfirmed.

> **Reading the comparison numbers.** GF is goals scored, GA is goals conceded, GD is GF minus GA. The attack rank orders goals scored per match from highest to lowest, the defense rank orders goals conceded per match from lowest to highest, both within one league and season, and tied rates share a rank. xG is not in the loaded data and cannot be derived from a scoreline.

How the team prepares

1. Each person studies their own card, does the exercises and finds the evidence in the repository.
2. Together, follow one path end to end: CSV → PostgreSQL → pipeline → API → screen.
3. Run the full 15-minute rehearsal with the timer and keep inside your own slot.
4. Each person answers at least two questions from another track.
5. Run the project on a second laptop and record the two-minute backup demo.

> **The split in this guide is a study and speaking plan.** It does not establish who wrote a given line of code. At the defense, describe the actual work you can show.

NEDALOZANOVSKA · 3:45 RECOMMENDED SPEAKING TIME

Data and the database

Scope: Sources → validation → bulk load → schema and SQL

What you need to know

- Which exact copy of the data we use. loader/sources.json pins one revision of the upstream repository — a revision is a specific saved version, like a dated edition of a report — and stores a SHA-256 checksum for every file. A checksum is a fingerprint of the file's contents: change one character and the fingerprint changes. That is how we prove we loaded the data we documented.
- One row of our analysis is one club, in one season, in one competition. The club's name is only a label. Two spellings of the same club must never become two clubs, so every join uses the numeric club ID, not the name.
- Why a relational (SQL) database: our eleven files have fixed columns, and we constantly combine players with clubs, seasons and competitions. That combining is called a join, and relational databases are built for it. A document database would store the same facts but make these joins harder, not easier.
- What COPY does: it is PostgreSQL's bulk-loading command. It streams a whole file into a table instead of sending one INSERT statement per row, which is why 5,743,186 rows load in about ten seconds.
- Why we build indices after loading, not before. An index works like the index at the back of a book: it makes lookups fast, but it must be updated for every row inserted. Building it once after the import is faster than maintaining it during the import. ANALYZE then refreshes the statistics the database uses to pick a query plan.
- The match files from football-data.co.uk are optional. Without them the system still runs and the interface states that coverage is missing instead of showing zeros. Club names in those files are mapped to our IDs, identical fixtures load once, and contradictory scores stop the replacement before stored matches change.
- The licence of the primary source is not confirmed. We checked: there is no licence file at the pinned revision. Being publicly downloadable is not permission to reuse. Say this plainly — it is a known open item, not a mistake to hide.
- One legacy table (team_competitions_seasons) differs from the current upstream file: 196,378 rows with seven fields locally, 58,247 rows with 29 fields upstream. We kept the local file instead of silently replacing it, and the Champagne Index and the model do not use it.

What to show

1. Open loader/sources.json on screen and read one entry aloud: the revision, the file name, its checksum and the expected column order.
2. Open sql/champagne_queries.sql and point at the GROUP BY: club ID, season, competition. Explain that MAX(team_name) only picks a label to display.
3. Open tests/test_match_loader.py and show one rejected case: a fixture whose stated result disagrees with its score never becomes a stored match.

Your spoken part

> *Our data layer uses eleven pinned CSV tables. We chose PostgreSQL because the records have fixed schemas and are joined by player, club, competition and season. The loader validates headers, uses COPY for bulk import, and builds indices afterwards. We aggregate by canonical identifiers so name aliases do not duplicate observations. An optional match source adds actual scorelines and standings. Invalid or duplicate matches are handled explicitly. The source is publicly downloadable, but its licence is still unconfirmed; we do not claim that this requirement is resolved.*

Hand over: “With the data stored consistently, the next step is to define a valid season-to-season target and evaluate it.”

Practice

1. Explain a checksum to someone outside the course in two sentences, without using the word hash.
2. Say why loading 5.74 million rows with COPY is faster than 5.74 million INSERT statements.
3. Describe what breaks if we join clubs by name instead of by ID, and give a concrete example from the data (Inter, Internazionale).

How to close the exam

1. You open the defense: slides 1–3, 00:00–03:45. You set the pace for the whole team, so speak slowly for the first twenty seconds.
2. Start with the question in plain language, then separate the two different questions immediately: what happens in the next season, and what we only observe within the same season.
3. Open loader/sources.json on the screen rather than describing it. One real manifest entry is more convincing than any slide.
4. Raise the licence yourself, before the examiner asks: the revision is pinned, the licence is not confirmed, and we do not redistribute the raw data.
5. Mention the legacy metadata table yourself too: one table differs from the current upstream version, and the central index and model do not use it.
6. Keep the database justification to one sentence: fixed columns and constant joins by player, club, season and competition.
7. Hand over with the prepared sentence instead of improvising — an improvised handover costs ten seconds you do not have.

Where to look in the project

Project file	What to look at in it
`loader/sources.json`	The pinned revision, file list, SHA-256 checksums and expected column order. Open this file during the defense instead of describing it.
`loader/get_data.py`	Header and checksum verification, download to a temporary file, atomic install. This is the answer to “what if the download is interrupted?”

Project file	What to look at in it
`loader/load_data.py`	Bulk COPY into PostgreSQL, match validation, parse_match_extras, and clearing stale match rows when the optional source disappears.
`sql/schema.sql`	Tables, indices and additive ALTER statements for the new match columns, so an existing database upgrades without losing loaded scores.
`sql/champagne_queries.sql`	The aggregation key club × season × competition, and MAX(team_name) used only as a display label.
`sql/team_name_map.sql`	Mapping of club names to canonical IDs before any match join, so alternative spellings cannot duplicate a club.
`tests/test_acquisition.py`	What the acquisition tests actually prove: a wrong checksum, a wrong header and an interrupted transfer install nothing.
`tests/test_match_loader.py`	Duplicate and contradictory fixtures, two- and four-digit years, and valid 0–0 draws.
`docs/SOURCES.md`	Provenance, the legacy metadata difference and the licence boundary — your answer to the licence question.

Phrasing to avoid

- Our data is open source.
- The database cleans the data for us.
- A zero transfer fee means the player moved for free.

LILLLAZ · 3:45 RECOMMENDED SPEAKING TIME

Аналитика и модель

Scope: Индекс → признаки → target → baseline → ограничения

Что нужно знать

- Формула Champagne Index; почему goal + assist может учитывать одно взятие ворот дважды и не является xG.
- Target = индекс следующего последовательного сезона минус текущий; минимум 30 матчей для топ-5 лиг и 5 для прочих соревнований.
- Пять признаков: текущий индекс, общие расходы, расходы на атаку, средний возраст, изменение средней рыночной стоимости.
- Как устраняется неверный лаг: уникальный клуб-сезон, точный предыдущий год, затем присоединение соревнований.
- Хронологический holdout: ранние сезоны обучают, поздние тестируют; baseline предсказывает нулевое изменение.
- R^2 = 0.127 не означает 87% или 12.7% точности. MAE 0.4856 против 0.5114 показывает небольшое улучшение.
- Lab использует отдельную модель на всех допустимых данных и не доказывает причинный эффект изменения бюджета.
- The season comparison also shows score-derived goals for/against/difference and per-match attack/defense ranks. These are separate from the Transfermarkt-based Champagne Index.
- The defensive-spend follow-up correlates defender fees with next-season change in goalkeeper-source goals conceded per match: r=0.0040, p=0.427, n=38,531; no useful linear signal.

Что показать

1. Показать текущие r, p, n и MAE на странице Project; назвать единицы и выборку каждого показателя.
2. Объяснить, почему 23/24 → 25/26 не является корректной парой, даже если shift(-1) её возвращает.
3. Поменять один ввод в Lab, вернуть baseline и объяснить, что измеряется сравнением сценариев.

Короткая речь на английском

> *We define attacking output as recorded goals plus assists per source match. The target is the change in that index in the next consecutive season. We remove incomplete season pairs and construct lagged squad values before joining competitions. Evaluation uses earlier seasons for training and later seasons for testing. The RandomForest has a test MAE of about 0.486, compared with 0.511 for predicting no change: an improvement of roughly five percent. This is a modest signal. The comparison also separates the Transfermarkt attacking index from completed league scores: goals for, goals against, their difference, and per-match attack and defense ranks. A defensive-spend follow-up also has almost zero association with next-season conceded-goal change (r=0.004, p=0.427; n=38,531). The Lab uses a separate all-data model for retrospective exploration. Moving a budget slider changes a model input; it does not establish what a transfer would cause.*

Переход: “The application makes those distinctions visible and lets us inspect the evidence for individual clubs.”

Практические задания

1. Рассчитай (76 + 70) / 38 и объясни, почему ответ 3.84 не равен голам за матч.
2. Посчитай (0.511364 − 0.485593) / 0.511364 ≈ 5.04% и назови правильную интерпретацию.
3. Различи три вопроса: корреляция в текущем сезоне, изменение в следующем сезоне и причинный эффект покупки игрока.

Как закрывать экзамен

1. Твои слайды 4 и 5 (03:45–06:30) и слайд 7 (09:15–10:15). Между ними 165 секунд демо у Джакоппо — следи за таймером, чтобы не потерять свой слайд findings.
2. Сначала определи наблюдение, потом показывай модель: клуб × сезон × соревнование, затем target как разницу с следующим последовательным сезоном.
3. Главное число произноси как сравнение, а не как достижение: MAE 0.4856 против baseline 0.5114 — улучшение около 5%.
4. Сразу объясни, что $R^2 = 0.127$ не доля правильных прогнозов. Это самый вероятный вопрос по твоей части.
5. Если спросят про Lab: отдельная модель на всех данных для исследования, её собственная подгонка не является независимым тестом.
6. Про любую связь говори «accompanies», «is associated with». Слово «causes» на твоей части стоит дороже всего.
7. Про xG отвечай коротко: поля провайдера нет, есть реальные удары и экспериментальный прокси по ударам в створ.

Где смотреть в проекте

Файл в проекте	Что в нём смотреть
`pipeline/compute_predictions.py`	Target, пять признаков, лаг рыночной стоимости, хронологический holdout и baseline нулевого изменения — всё в одном файле.
`docs/LAB.md`	Протокол модели и отличие Lab-модели (все данные) от оценочной модели (ранние сезоны → поздние).
`docs/findings.md`	Актуальные r, p, n, R^2 и MAE. Называй числа отсюда, а не по памяти.
`docs/GLOSSARY.md`	Определения Champagne Index, Discipline Index, рангов и граница xG.
`pipeline/feature_correlations.py`	Матрица корреляций признаков и флаг коллинеарности |r| > порога.
`pipeline/shap_analysis.py`	mean |SHAP| на том же held-out сплите, что и R^2 — ответ на «почему не in-sample».
`pipeline/power_ratings.py`	Poisson attack/defense с общим home advantage; в коде прямо написано, что оценка in-sample.
`pipeline/proxy_xg.py`	Прокси по ударам в створ и finishing/defensive luck — почему это не xG.
`pipeline/discipline_effect.py`	Связь красных карточек с пропущенными: +0.341 гола за матч, 1 619 против 16 195 наблюдений.
`tests/test_regressions.py`	Зафиксированные известные дефекты модели и расчётов, чтобы они не вернулись.

Какие фразы не использовать

- Модель доказывает, что увеличение бюджета улучшит игру.
- R^2 — доля правильных прогнозов.
- Оценка отдельной holdout-модели является результатом тестирования all-data модели Lab.

JACOPOSCACCIANOCE · 3:45 RECOMMENDED SPEAKING TIME

The application and the demo

Scope: API → page states → comparison → scenarios → live demo

What you need to know

- What an API is here: a small set of web addresses (endpoints) that return prepared results as JSON — plain text data that a browser can read. The browser asks, the server answers with a few hundred rows, never with the millions of raw rows in the database.
- Why that matters: the SQL, the validation and the credentials stay on the server. The page only displays what the server already prepared and checked.
- The numbers of your own demo pair, so you never read them off the screen for the first time while presenting: in Serie A 24/25 Inter recorded 79 GF, 35 GA and +44 GD (attack rank 1, defense rank 2) and Milan 61, 43 and +18 (ranks 3 and 7); their two meetings ended 1–2 and 1–1, so the head-to-head total is 2–3.
- The three states that are easy to confuse: no coverage for this league and season, coverage exists but the two clubs never met, and a real completed 0–0 draw. The interface shows a different message for each and never invents a score.
- What the HTTP status codes mean in our app: 422 means the input is invalid, for example the same club selected twice; 404 means the requested club or season is not in scope; 503 means the data or the model is not ready yet, which is a readiness problem, not a bad request.
- How head-to-head goals are counted: for each completed meeting we take the selected club's perspective, so its goals are the home score when it played at home and the away score otherwise. Swapping the two clubs swaps the totals. Wins, draws and points follow from those scores.
- What protects the Lab inputs: the server rejects values that are not finite numbers, checks the allowed range and refuses an attacking budget larger than the total budget. The sliders also limit the range, but the server check is the one that matters, because anyone can call the API directly.
- Every new field reads from a table only after checking that the table exists. If the optional match data was never loaded, the API answers “absent” instead of failing — that is why the demo survives a partial setup.
- What the automated checks cover: 43 unit tests, 16 live API checks and 15 real-browser checks, including mobile layout at 390 pixels. They are scoped evidence, not proof that no bug exists.

What to show

1. Open /api/docs and call one endpoint live, then show the same numbers on the page: the page is a view of the API, not a separate truth.
2. Open a comparison for two clubs in a season with match data, then switch to a league-season without it and read the coverage message aloud.
3. In the Lab, move one input, show the new prediction, then press Reset and explain that a slider changes a model input, not reality.

Your spoken part

> *The browser reads a bounded API over prepared results. League and season filters are explicit, and the URL preserves the selected scope. In the comparison, season totals and actual head-to-head meetings are separate. Serie A 24/25 shows Inter at 79 for and 35 against (+44), ranked first in goals scored per match and second in goals conceded per match. Milan has 61 for and 43 against (+18), ranked third and seventh. The source has no xG field. For Inter and Milan in 24/25, the recorded league scores are 1–2 and 1–1, giving head-to-head goals of two to three. Missing match enrichment is reported explicitly. The Lab loads the five real club-season inputs, calls a saved model and rejects invalid budgets or non-finite numbers. We verify behavior at three levels: regression tests, live data contracts and desktop/mobile browser checks.*

Hand over: “The final question is whether this system runs reproducibly and what resources it uses.”

Practice

1. Explain the difference between 422 and 503 to someone who has never used an API, in one sentence each.
2. Say out loud why “no recorded meeting” and “0–0” must look different on the screen.
3. Call /api/head-to-head for two clubs from the same city and verify that reversing the clubs swaps goals for and goals against.

How to close the exam

1. You own slide 6 (the live demo, 06:30–09:15) and slide 10 (the close, 14:00–15:00). The demo is the longest single block and the most fragile one.
2. Follow the five demo steps in order and say what you expect to appear before it appears. That turns a loading delay into a prediction instead of an awkward silence.
3. If something is not up, do not debug Docker in front of the examiner. One sentence — “we move to the recorded evidence” — then use the screenshots in defense/site/assets.
4. Show the missing-coverage state on purpose. It proves the interface does not invent a 0–0 result.
5. In the Lab change exactly one input, then reset. One factor at a time is the same discipline we use in the measurements.
6. If asked why an API instead of plotting the tables directly: millions of rows never travel to the browser, and SQL plus validation stay on the server.
7. You speak last. Name the project's boundaries and the open items yourself — the examiner hears honesty as strength, not as weakness.

Where to look in the project

Project file	What to look at in it
`app/api.py`	Endpoint contracts, the 422 / 404 / 503 answers, bound parameters, read-only transactions and the table-existence guard behind every new field.

Project file	What to look at in it
`app/static/app.js`	The page states: missing coverage, no recorded meeting and a real 0–0 are three different outcomes.
`app/static/api-docs.html`	The API documentation page you open during the demo at /api/docs.
`scripts/check_live.py`	The 16 live checks of the API contracts and the prepared outputs; run it after the stack is healthy.
`scripts/browser_check.mjs`	The 15 real-browser checks of actual flows and the mobile layout; screenshots land in artifacts/.
`scripts/site_check.mjs`	The nine checks of the offline study site; the result is written to defense/site-check.json.
`tests/test_head_to_head.py`	Proof that goals for and goals against swap when the two compared clubs are reversed.
`tests/test_health.py`	Why 503 means “data or model not ready” rather than “bad request”.
`docs/FRONTEND_HANDOFF.md`	The data contract for any other frontend — evidence that presentation is separated from processing.

Phrasing to avoid

- Missing data is the same as zero.
- The sliders in the Lab show what a transfer would cause.
- The tests prove the application has no bugs.

Инфраструктура и измерения

Scope: Compose → readiness → воспроизводимость → время/RAM/диск/рост

Что нужно знать

- Порядок db healthy → loader success → pipeline success → app. Почему depends_on с успешным завершением отличается от простого запуска процесса.
- Named volume базы и bind mounts для CSV, модели и измерений; почему секреты и сырые данные исключены из Git и Docker build context.
- Методика: три повторения, среднее и отдельные результаты, одна изменяемая величина, характеристики ноутбука.
- Полный запуск 31.10 s относится к готовым CSV и образам; это не время первого скачивания и сборки.
- Индекс сравнивается на одной выборке 300 клубов; 25/50/100% эксперименты — отдельная агрегация крупнейшей таблицы.
- Память — максимальное наблюдаемое working set из docker stats, а не непрерывно измеренный истинный пик.
- Что нужно подготовить к защите: данные и образы заранее, рабочий ноутбук, сохранённые результаты, PDF и двухминутное резервное видео.

Что показать

1. Открыть docker-compose.yml и проследить зависимости и mounts без публикации паролей.
2. Показать README/MEASUREMENTS.md: машина, три прогона, диапазон условий и рост данных.
3. Объяснить отличие 31.10 s запуска от 10.872/0.0748 s запросов и от времени импорта одного CSV.

Короткая речь на английском

> *Docker Compose coordinates four core services. The database must be healthy, loading and processing must finish successfully, and then the application starts. Source files, the model and measurement results use explicit mounts. We measured timing, observed memory, disk footprint and growth in separate experiments, each repeated three times. Complete cached-source startup averaged 31.10 seconds on an Apple M3 Pro with 18 GiB RAM. The selective index lookup improved by about 145 times, but that result applies to that query and sample. Caches were retained, image builds were excluded and brief memory peaks may have been missed.*

Переход: “Taken together, the system is reproducible on the measured setup, while the data and model limitations remain explicit.”

Практические задания

1. По схеме объясни, что произойдёт, если loader завершится с ошибкой, а база уже healthy.
2. Сравни 25% и 100%: строк в 4 раза больше, загрузка примерно в 4.40 раза дольше; объясни, почему это не универсальный закон.
3. Подними проект по README на другом ноутбуке и запиши наблюдения. Не объявляй эту проверку выполненной до реального запуска.

Как закрывать экзамен

1. Твои слайды 8 и 9 (10:15–14:00) — ты закрываешь содержательную часть, последнюю минуту берёт Джакоппо, и твоя часть прямо соответствует обязательному пункту «производительность в числах».
2. Каждое число произноси с условиями: 31.10 с — среднее трёх прогонов с готовыми источниками и собранными образами, без скачивания и без сборки.
3. 145.4× называй как результат одного эксперимента на 300 выборочных запросах, а не как ускорение всего приложения. Этот вопрос задают почти всегда.
4. Память описывай методом: максимальная наблюдённая выборка docker stats с интервалом около двух секунд, не гарантированный пик.
5. Рост 25 / 50 / 100% объясняй нелинейностью: 4× строк дали 4.40× загрузки и 3.35× агрегации из-за фиксированных затрат, планов и кэшей.
6. Меняй один фактор за раз — скажи это вслух, это прямое требование протокола измерений из брифа.
7. В конце назови, что осталось: лицензия, независимый запуск на другом ноутбуке, резервное видео и фактический состав участников. Не заявляй их выполненными.

Где смотреть в проекте

Файл в проекте	Что в нём смотреть
`docker-compose.yml`	Порядок запуска: healthcheck базы → успешный loader → успешный pipeline → приложение. Профили для measure и legacy UI.
`docs/MEASUREMENTS.md`	Три прогона, железо (M3 Pro / 18 GiB), метод выборки памяти и все оговорки. Главный документ твоей части.
`docs/ARCHITECTURE.md`	Контейнеры как блоки, стрелки как файлы, строки, SQL и HTTP — структура для слайда архитектуры.
`docs/COURSE_REQUIREMENTS.md`	Таблица требований курса со страницами брифа и статусом каждого пункта.
`measure/benchmark.py`	Эксперимент с индексом: 300 одинаковых выборочных запросов, 145.4× — и почему это не ускорение всего приложения.
`scripts/measure_system.py`	Полный прогон с нуля: старт 31.10 с, loader 12.53 с, pipeline 9.7 с (средние трёх прогонов).

Файл в проекте	Что в нём смотреть
`scripts/report_measurements.py`	Сборка измерений в отчёт — показывает, что числа не вписаны руками.
`docs/AUDIT.md`	Найденные и исправленные дефекты: ответ на вопрос «как вы убедились, что это работает».
`docs/WORK_PACKAGES.md`	Пакеты работ, приёмочные условия и честный список незакрытых внешних пунктов.
`scripts/build_defense.py`	Сборка этого пособия: слайды, роли, PDF и проверка, что выступление ровно 900 секунд.

Какие фразы не использовать

- 31 секунда включает скачивание 353 MiB и сборку образов.
- docker stats гарантирует истинный максимальный расход памяти.
- Одинаковое число коммитов подтверждает одинаковый личный вклад.

Plain-language glossary

Short, non-technical definitions of every term the examiner may use.

Container / Docker Compose	A container is one packaged program with everything it needs to run. Compose is the file that starts our four containers in the right order: the database first, then the loader, then the processing pipeline, then the web application.
Relational database / PostgreSQL	A store of tables with fixed columns that can be combined by shared keys. We use PostgreSQL 16. Combining two tables on a shared key is called a join.
Schema	The declared shape of the tables: which columns exist and what type each one holds. It is written in sql/schema.sql.
COPY and index	COPY loads a whole file into a table in one stream. An index is a lookup structure, like the index of a book: it makes selective queries fast and is built after the bulk load.
Pipeline	The processing step between the database and the web page. It reads tables, computes our measures and the model, and writes prepared result tables back.
API / endpoint / JSON	The API is the set of web addresses the page calls. Each address is an endpoint. JSON is the plain-text format of the answer.
Champagne Index	Our own measure of attacking output: recorded goals plus assists, divided by the number of matches the source reports for that club-season. It is a proxy, not an official statistic.
Target and feature	The target is what the model tries to predict — here the change in the index in the next consecutive season. A feature is an input the model uses to predict it.
Chronological holdout	We train the model on earlier seasons and test it on later seasons it has never seen. A random split would mix the future into the training data and flatter the result.
Baseline	The simplest possible prediction, used for comparison. Ours predicts no change at all. A model is only useful if it beats the baseline on the same test rows.
R^2 and MAE	R^2 is the share of variation the model explains, from 0 to 1 — ours is 0.127, which is weak. MAE is the average absolute error in index units — ours is 0.4856 against a baseline of 0.5114, about 5% better. Neither is a percentage of correct answers.
Correlation vs. causation	A correlation (r) measures how two numbers move together, from −1 to +1. It cannot show that one causes the other. Every result in this project is a correlation, including the red-card and spending findings.
Proxy	A measurable stand-in for something we cannot measure directly. Our shots-on-target estimate is a proxy for chance quality; it is not xG, which the data source does not contain.
Poisson rating	A simple statistical model of how many goals each team scores and concedes, fitted to one league-season. It describes that season; it does not forecast new matches.

SHAP	A method that attributes a model's prediction to its individual inputs, so we can report which feature mattered most without relying on the model's internal importance score.

Examiner questions

Answer out loud in one short sentence first, then give the evidence and the limitation.

NEDALOZANOVSKA

Why PostgreSQL rather than MongoDB?

The source tables have fixed columns and relational links by player, club, season and competition. PostgreSQL supports those joins, native COPY and indexed lookups with a clear schema. A different database could work, but this choice follows the data shape.

NEDALOZANOVSKA

What identifies one observation?

For the analytical output it is club ID, season and competition ID. A club's display name is a label, not its identity. We aggregate canonical keys to prevent alias names from creating duplicate observations.

NEDALOZANOVSKA

Why build indices after loading?

Maintaining indices during a bulk load adds work to every inserted row. We import with COPY, then build the required indices and ANALYZE the updated tables so the next pipeline has current planner statistics.

NEDALOZANOVSKA

Is your source openly licensed?

We verified the pinned source revision, headers and checksums, but did not establish an explicit upstream licence. We document this as unresolved and exclude raw data from the repository. Public downloadability alone is insufficient.

NEDALOZANOVSKA

What happens if a download is interrupted?

The downloader writes a temporary file and installs it atomically only after checksum and header validation. An interrupted or corrupt download is not accepted as a complete source. Tests exercise these failures using fake HTTP responses.

NEDALOZANOVSKA

How do you handle duplicate match rows?

Identical league/season/date/home/away fixtures are imported once. Conflicting scores for the same fixture stop replacement before stored match rows are changed. Missing optional input clears old enrichment on the next loader run.

LILLLAZ

Is Champagne Index an official metric or xG?

Это официальный индекс или xG?

No. It is this project's proxy: recorded goals plus assists divided by source matches. A goal with an assist contributes twice. We explain the denominator and do not present it as xG or an official rating.

LILLLAZ

What exactly does the model predict?

Что именно предсказывает модель?

The next consecutive season’s Champagne Index minus the current index. It does not predict a match score, a calibrated league position or the causal return from buying a player.

LILLLAZ

Why reject gaps between seasons?

Почему нельзя брать просто следующую строку?

A club may leave a competition and return years later. shift(-1) gives the next recorded row, which is not necessarily next year. We explicitly require a one-year gap and adequate coverage in both seasons.

LILLLAZ

Why use a temporal split?

Зачем хронологический holdout?

A random row split mixes earlier and later seasons and can make historical evaluation overly optimistic. We train on earlier seasons and test on later ones, keeping observations from the same season together.

LILLLAZ

How much better is the model than a baseline?

Насколько модель лучше baseline?

On the same 5,585 held-out rows, MAE is about 0.4856 versus 0.5114 for predicting no change. That is about a five percent reduction in MAE, a modest gain. R-squared is about 0.127, not a percentage of correct predictions.

LILLLAZ

Can you conclude that buying attackers improves performance?

Можно ли утверждать причинный эффект?

No. These are retrospective observational data with confounding and source limitations. A slider changes a model input and shows a learned association, not the outcome of a controlled transfer intervention.

LILLLAZ

Why is the Lab model different from the evaluation model?

Почему две обученные модели?

The evaluation model uses only earlier training data and is scored on held-out later seasons. A separate model is refitted on all eligible data for exploration. Its own training fit is not claimed as independent test performance.

LILLLAZ

Why did the index-position correlation change after the audit?

Почему изменилась корреляция?

The earlier result mixed a partial 25/26 attacking snapshot with full final standings. We now require adequate comparable coverage. The corrected same-season association is about minus 0.793 over 388 observations.

JACOPOSCACCIANOCE

Why have an API instead of plotting raw tables directly?

The API exposes small prepared outputs under an explicit league and season. This avoids moving millions of raw rows to the browser, keeps SQL and validation on the server and separates presentation from processing.

JACOPOSCACCIANOCE

How are head-to-head goals calculated?

For each completed meeting, the selected club's perspective determines whether the home or away score is goals for. Reversing the compared clubs swaps their totals. Wins, draws, losses and points are derived from those actual scores.

JACOPOSCACCIANOCE

Does no recorded match mean a 0–0 draw?

No. No coverage, no recorded meetings and a completed 0–0 draw are different states. The API reports enrichment availability and the interface shows an explicit message instead of inventing a score.

JACOPOSCACCIANOCE

What prevents invalid scenario inputs?

The API rejects non-finite numbers, checks bounds and requires attacking spend not to exceed total spend. The browser also clamps the sliders. Server-side validation remains necessary even when the UI enforces the same rule.

JACOPOSCACCIANOCE

What do HTTP 422, 404 and 503 mean here?

422 indicates invalid input, such as selecting the same club twice. 404 means the requested club-season scope is unavailable. 503 indicates temporary data/model unavailability or an incomplete application output state.

JACOPOSCACCIANOCE

What do your tests prove?

Regression tests cover known failure cases, live checks validate the prepared outputs and API contracts, and browser checks exercise actual flows and mobile layout. They provide scoped evidence, not a guarantee that every possible bug is absent.

ATAIKA

What starts when you run Docker Compose?

Что запускает Compose?

The database becomes healthy, then the loader must finish successfully, then the processing pipeline must finish, and finally the application starts. The measurement service is on demand and the legacy interface uses an optional profile.

ATAIKA

What does the 31.10-second measurement include?

Что входит в 31.10 секунды?

A mean over three complete starts with prepared source files and existing images: containers are recreated, sources reimported and the model retrained until the API is ready. Network acquisition and image builds are excluded and caches are not flushed.

ATAIKA

Does the index make the whole application 145 times faster?

Всё приложение быстрее в 145 раз?

No. The measured gain is for 300 identical selective club lookups on the raw performance table, comparing index conditions. The frontend reads prepared outputs and a full GROUP BY has a different access pattern.

ATAIKA

Is your memory number a guaranteed peak?

Это гарантированный пик RAM?

It is the largest observed Docker working-set sample during three runs, sampled at roughly two-second intervals. Brief peaks may be missed. We report the method rather than label it a continuous or hardware-independent maximum.

ATAIKA

Why did four times as many rows not give exactly four times the query time?

Почему масштабирование не точно линейное?

Fixed costs, query plans, indexing, caches and hardware influence scaling. In this experiment four times the largest-table rows gave about 4.40 times the loading time and 3.35 times the aggregation time. That does not define a general law.

ATAIKA

What remains before a group submission?

Что ещё нужно для сдачи?

Confirm actual team membership and each person’s contribution, run an independent second-laptop setup, resolve the source licence, rehearse the presentation and record a backup demo. Equal author email counts alone do not demonstrate real group participation.

EVERYONE

What is the main result in one sentence?

Главный результат одним предложением?

The system is reproducible on the measured setup, while historical transfer spending alone has almost no linear relationship with next-season index change and the richer model adds only modest predictive value.

EVERYONE

What is the most important limitation?

Главное ограничение?

The data are retrospective and unevenly covered, the index is a proxy, and associations are not causal effects. Source licence and independent team/laptop validation also remain explicit unresolved requirements.

EVERYONE

How do goals for, goals against and goal difference work in the comparison?

Как в сравнении рассчитываются забитые, пропущенные голы и разница?

GF and GA are season totals rebuilt from completed home/away match scores. GD is GF − GA. They come from the optional football-data.co.uk match enrichment, not from the Champagne Index player totals.

EVERYONE

How are attacking and defensive ranks calculated?

Как рассчитываются атакующий и защитный рейтинги?

Within one league and season, attack rank orders goals scored per match from highest to lowest; defense rank orders goals conceded per match from lowest to highest. SQL RANK gives tied rates the same rank and leaves a gap after ties. They describe scores, not tactics or chance quality.

EVERYONE

Do we have xG or can we estimate it?

Есть ли в данных xG или можем ли мы его оценить?

The source has no provider xG field. The comparison instead shows observed shots and shots on target. A separate estimate multiplies shots on target by the league-season average goals-per-shot-on-target rate; it is an experimental volume proxy with no shot-location or chance-quality input, not xG.

EVERYONE

What is the difference between the Champagne Index and attack rank?

Чем атакующий рейтинг отличается от Champagne Index?

The Champagne Index is recorded player goals plus assists divided by a Transfermarkt matches proxy for a club-season. Attack rank uses actual completed league scorelines, sums team goals and ranks goals per match within the league-season. Their sources, coverage and units differ.

EVERYONE

How did you check the goalkeeper defensive totals?

Как вы проверили вратарские показатели по пропущенным?

We compared Transfermarkt goalkeeper goals-conceded totals with the independent match-result totals on 388 adequately covered top-five club-seasons. They match exactly in 366 (94.3%). Clean sheets match only 269 (69.3%), so the scoreline source remains the basis of league ranks.

EVERYONE

What does the defensive-spend analysis show?

Что показывает анализ расходов на защитников?

The Pearson correlation between defender transfer fees and next-season change in the goalkeeper-source conceded-goal rate is r=0.004047, p=0.426997, n=38,531. The sample does not support a useful linear association. It is descriptive, not causal. A separate match-level check finds teams with a red card concede 0.341 more goals on average in the same match, but match context confounds that association.

EVERYONE

Why keep the player-source defensive values separate from scoreboard goals?

Почему defensive-показатели по игрокам отделены от итогов матчей?

The sources use different records and denominators. Goalkeeper conceded totals agree with scorelines in 94.3% of adequately covered seasons, while clean sheets agree in 69.3%. The UI therefore uses match scores for GF, GA, GD and ranks; player-source totals remain a quality cross-check. Shots and corners are also observed match statistics, not chance-quality measures.

EVERYONE

Is the shot-based estimate the same as xG?

Оценка по ударам — это xG?

No. It multiplies a team's shots on target by the league-season average goals per shot on target. It has no shot-location, distance, body-part, pressure or goalkeeper-position input. It is a rough volume benchmark and must be called an SOT-based proxy, not provider xG.

EVERYONE

What does the Poisson attack/defense rank tell us?

Что показывают Poisson-ранги атаки и обороны?

The model fits attack, defense and a shared home-advantage term to the selected league-season scores. Larger fitted attack and defense parameters mean stronger estimated attack and defense. Its ranks are descriptive and in-sample; the separate chronological RandomForest evaluation is the project's held-out prediction check.

EVERYONE

Did the added defensive features improve the model?

Улучшили ли защитные признаки модель?

On the same chronological split, R^2 moves from 0.1271 with the five-feature baseline to 0.1502 after adding goalkeeper conceded rate, discipline and net spend; MAE moves from 0.4856 to 0.4788 on 5,585 held-out rows. The gain is modest, and it does not isolate a causal effect of defense.

EVERYONE

Do red cards cause more goals to be conceded?

Красные карточки приводят к большему числу пропущенных?

The local sample shows a same-match difference of +0.341 conceded goals per team-match (1,619 sent-off vs 16,195 others; Welch $p < 0.0001$). This is observational: stronger opponents, match state and timing can affect both red cards and goals. It is not a causal estimate.

The 15-minute running order

Slide	Owner	Time	What to show
1. The price of beautiful football	nedalozanovska	00:00–01:00	Does incoming transfer spend accompany better attacking output next season?; Separate question: does the index track same-season league finish?; A reproducible data system with measured infrastructure.
2. Know the source	nedalozanovska	01:00–02:30	11 pinned CSV tables · 5.74m local rows; Optional 8,908 completed matches across five leagues; Headers, hashes and atomic acquisition; raw data excluded from Git; Boundary: source licence still unconfirmed.
3. From CSV to the screen	nedalozanovska	02:30–03:45	GET → STORE → PROCESS → SHOW; PostgreSQL 16 + loader + Python pipeline + FastAPI; Compose waits for health and successful completion; Model and measurement artifacts use explicit mounts.

Slide	Owner	Time	What to show
4. Define the observation before fitting	lilllaz	03:45–05:15	Club × season × competition is one observation; Index = (recorded goals + assists) / source matches; Target = next consecutive season index − current index; Exclude inadequate season pairs and competition-unsafe lags.
5. An honest model comparison	lilllaz	05:15–06:30	Earlier seasons train; later seasons test; Test MAE 0.4856 vs zero-change baseline 0.5114; About 5% less absolute error; R^2 = 0.127; Separate all-data model powers retrospective scenarios.
6. See the evidence	jacoposcaccianoce	06:30–09:15	Serie A 24/25 → two clubs → comparison; Season GF / GA / GD: Inter 79 / 35 / +44; Milan 61 / 43 / +18; Attack ranks 1 / 3; defense ranks 2 / 7, by per-match goals; Separate head-to-head: scores 1–2 and 1–1; aggregate goals 2–3; Open Lab → change one input → reset; Project page: sample sizes, methods and measurements.; Shots / shots on target / corners: coverage 8,907 of 8,908 matches; Poisson model ranks are in-sample; the SOT conversion proxy is not xG
7. What the data supports	lilllaz	09:15–10:15	Transfer spend → next attacking change: r = −0.0036; n = 38,531; Defender spend → next conceded-goal change: r = +0.0040; p = 0.427; Same-season index ↔ position: r = −0.793; n = 388; Different targets; these are associations, not interventions.; Same temporal split: R^2 0.127 → 0.150 with conceded rate, discipline and net spend; Red-card difference +0.341 conceded goals; observational, not causal
8. Measure the whole system	Ataika	10:15–12:30	Three repetitions on Apple M3 Pro / 18 GiB; Cached-source startup 31.10 s; 300 lookups: 10.8724 s → 0.0748 s with index; Database 695.5 MiB; source CSVs 356.6 MiB.
9. Growth, memory and failure cases	Ataika	12:30–14:00	25/50/100% of the largest table; 4× rows: 4.40× load time; 3.35× query time; Observed per-container memory, not guaranteed instantaneous peak; 38 regression + 16 live + 15 browser checks.
10. Boundaries and ownership	jacoposcaccianoce	14:00–15:00	Data quality and causal interpretation stay explicit; Source licence and independent laptop validation remain open; Explain the actual work of each real participant; Take questions on any part of the system.

Demo and readiness

Time	Action	Expected / fallback
Before speaking	docker compose up -d; open /api/health and preload the app	ready=true; app/db healthy; loader/pipeline completed Use saved screenshots and the PDF presentation; do not troubleshoot Docker for the whole presentation.

Time	Action	Expected / fallback
0:00–0:30	Open Overview, Serie A 24/25	20 clubs and a complete-source season Open overview-desktop.png from the evidence folder.
0:30–1:15	Open Inter/Milan comparison	Two recorded meetings (2–3 head-to-head goals) and season GF/GA/GD with league ranks If match source is absent, explain the coverage message and use comparison-desktop.png as the recorded example.
1:15–2:15	Open Lab, change only total spend, then Reset	Scenario response and baseline return; attack budget remains within total Show lab-desktop.png and explain saved-model inference.
2:15–3:00	Open Project → findings / measurements	r, n, MAE/baseline and dated timing evidence with limitations Use the results and measurement slides.

Before submission

- Everyone can walk through GET → STORE → PROCESS → SHOW without prompting
- Everyone has worked through their own track, its exercises and its questions
- Everyone has answered at least two questions from outside their own track
- The presentation fits into 15 minutes and every real participant speaks
- Another participant ran a fresh checkout from the README alone and recorded the result
- The primary source's licence is confirmed, or a clearly licensed source is chosen instead
- The two-minute backup recording exists and has been played back once
- The actual participants and the real changes and checks each of them made are stated

Exam day

Date	Format	What is required
12 November 2026 CIDS Lecture 15, pp. 37–39	Checkpoint · 5 minutes	One end-to-end demo, one visible result and the first measured full-run time. A polished interface is not required.
20 / 26 November 2026 CIDS Lecture 15, pp. 37–39	Final defense · 15 minutes + 5 minutes of questions	Every real participant speaks. Suggested split: question and data 3 min, architecture 2, live demo 3, measurements 3, problems 2, contributions 2.

Machine and materials

1. Thirty minutes ahead: run `docker compose up -d --build`, then open /api/health and wait for `ready=true` with the loader and pipeline finished.
2. Open four tabs in advance: Overview (Serie A 24/25), the Inter / Milan comparison, the Lab, and the Project page with the numbers.
3. Open the offline study site `defense/site/index.html` and `defense/site/downloads/Presentation.pdf`: both work without a network and without the database.
4. Check that the screenshots in `defense/site/assets/*-desktop.png` are present. They are the fallback if Docker does not come up.
5. Turn off notifications, connect the charger, test screen mirroring and increase the browser font size.
6. Decide in advance who drives the demo and who watches the clock. The timer is built into `defense/site/presentation.html` (N for notes, F for fullscreen).

How to answer questions

- Answer in three steps: one short sentence, then the evidence (a file or a number), then the limitation.
- Give every number with its sample and unit: “r = −0.79 across 388 club-seasons, within the same season”, not “the correlation is high”.
- If you do not know, say where the answer lives instead of guessing: “that is in docs/MEASUREMENTS.md, the memory sampling method is described there”.
- If the question belongs to someone else's part, hand it over in one sentence rather than approximating their area.
- Never turn an association into a cause. Say “accompanies” or “is associated with”, never “causes” or “improves”.
- If asked about a weak point, answer it first and directly. An acknowledged limitation is stronger than a defended inaccuracy.

Uncomfortable questions: answer them first

Topic	Question	Answer and evidence
Licence of the primary source	Is your data openly licensed?	We pinned the source revision and verified headers and checksums, but there is no licence file, so we did not establish an explicit reuse licence. We document it as unresolved and we do not redistribute the raw data. Public downloadability is not a licence. `docs/SOURCES.md, docs/LICENCE_DECISION.md`
Four contributors	Who did what in this project?	Describe only the work you can open and explain on screen. Git identities and role cards are not proof of independent contribution, and the repository says so. `docs/WORK_PACKAGES.md, README.md`
Reproduction on another machine	Did someone else reproduce it?	The run is reproducible on the measured machine, and we also rebuilt it from a clean clone. An independent fresh checkout on another person's laptop has not been recorded yet, so we do not claim it. `docs/FRESH_CHECKOUT.md, docs/MEASUREMENTS.md`
xG and proxies	Is this xG?	No. The source has no provider xG field. We show observed shots and shots on target, and a shots-on-target conversion proxy that we label experimental. The Poisson attack and defense ratings describe the season they are fitted on. `docs/GLOSSARY.md, pipeline/proxy_xg.py, pipeline/power_ratings.py`

Checked facts and their limits

Measure	Value	Explanation
Primary source rows	5,743,186	11 local CSV tables; one legacy metadata table differs from the fresh normalized source.
Completed match enrichment	8,908	Optional, five leagues and seasons 21/22–25/26. Not needed for the primary pipeline.
Spend → next-season change	r = −0.003613	p = 0.478242; n = 38,531 eligible consecutive pairs. A practically tiny association.
Index ↔ final position	r = −0.793285	n = 388 comparable club-seasons. Same-season association; excludes partial index snapshots.
Temporal test R^2	0.127115	Train: 16,074 earlier rows; test: 5,585 rows from season-start year 2021.
Model MAE / baseline MAE	0.485593 / 0.511364	About 5.0% lower MAE. Baseline predicts zero next-season change on the same test rows.
Complete cached-source start	31.10 s	Mean of three runs on Apple M3 Pro / 18 GiB. Download and image build excluded; caches retained.

Measure	Value	Explanation
Selective index lookup speedup	145.4×	300 identical club lookups: 10.8724 s without index, 0.0748 s with index. Not every query gets this gain.
Score-derived league comparison	GF / GA / GD + 2 ranks	Serie A 24/25: Inter 79/35/+44 · attack #1 · defense #2; Milan 61/43/+18 · #3/#7. Per-match goals, league-season ranks.
xG coverage in loaded match source	Not available	No xG field exists in the loaded match-results schema. We do not infer expected goals from final scores.
Goalkeeper GA matches scoreboard	366 / 388 exact · 94.3%	Adequately covered top-five club-seasons; Transfermarkt goalkeeper totals cross-checked against actual completed-match league scores.
Clean sheets match scoreboard	269 / 388 exact · 69.3%	Primary-source clean-sheet totals are less consistent, so the comparison does not use them as a league defensive rank.
Defensive spend → next GA change	r = 0.004047	p = 0.426997; n = 38,531 eligible consecutive pairs. No useful linear association in this snapshot.
Shot and corner coverage	8,907 / 8,908 matches	Shots, shots on target and corners; team comparison uses the exact available-match denominator.
Poisson attack/defence fits	484 team-seasons · 25 league-seasons	Descriptive ratings fitted on the same season scores; not an out-of-sample forecast.
Attack-only → augmented model R^2	0.1271 → 0.1502	Same chronological split and 5,585 held-out observations; augmented features add conceded-goal rate, discipline and net spend.
Red card association	+0.341 conceded goals / team-match	1,619 sent-off vs 16,195 other team-matches; Welch $p < 0.0001$. Descriptive and confounded, not causal.
SHAP · leading feature	Champagne Index · 0.205 mean |SHAP|	Held-out augmented model. Explains this fitted model, not football causally.

The primary source is pinned to a revision, but its licence is not confirmed. That is an open item, and a link to a public download does not close it.